

Visual Product Search Engine

Final Project Report

Team Members

Sarthak Maheshwari	IMT2023014
Chandrabhas Reddy	IMT2023037
Harsh Mohta	IMT2023106

Course: Visual Recognition

Dataset: DeepFashion In-Shop Clothes Retrieval

[Github](#)

Abstract

We present a query-by-image product search system that enables users to find visually and semantically similar fashion items from the DeepFashion In-Shop Clothes Retrieval benchmark [1]. Our pipeline combines object detection (YOLOv8), semantic captioning (BLIP-2), cross-modal embedding fusion (CLIP), approximate nearest-neighbor indexing (HNSW), and BLIP-2 image-text matching re-ranking. Through a three-way ablation study we demonstrate that fine-tuning the CLIP vision encoder with a contrastive objective yields the largest single improvement, and that image-heavy fusion ($\alpha = 0.7$) outperforms text-heavy fusion ($\alpha = 0.3$) once the encoder is task-adapted. Our best configuration achieves **Recall@15 = 0.5002**, **NDCG@15 = 0.3996**, and **mAP@15 = 0.2842** — roughly a 2× improvement over the vision-only CLIP baseline on all metrics.

Contents

1	Introduction	3
2	Problem Statement	3
3	System Architecture	3
3.1	Offline Indexing Pipeline	4
3.2	Online Query Pipeline	4
4	Fine-Tuning Strategy	5
4.1	Overview	5
4.2	Objective	5
4.3	What We Update	5
4.4	Reproducibility	6
5	Evaluation Protocol	6
5.1	Metrics	6
5.2	Ablation Conditions	6
6	Experimental Results	6
6.1	Main Results	6
6.2	Analysis	7
6.3	Visual Analysis of Results	8
6.4	Qualitative Results	10
7	Implementation Details	11
7.1	Models and Libraries	11
7.2	Compute Environment	11
7.3	Caching	11
8	YOLOv8 Fine-Tuning for Clothing Localization	11
8.1	Motivation	11
8.2	Class Definition	11
8.3	Dataset Preparation	12
8.4	Training Configuration	12
8.5	Results	12

9 Interactive Demo Application	13
9.1 End-to-End Query Flow	13
9.2 Cross-Class NMS in the Detection Module	13
9.3 Configuration Options	14
10 Inference Script	14
11 Discussion	14
12 Conclusion	15

1. Introduction

Online shoppers frequently encounter a simple yet persistent problem: they see a product image they like but cannot locate similar items through conventional text search. Text-based retrieval systems suffer from two fundamental failure modes. First, there is a *language mismatch* between user queries (*e.g.*, “black hoodie”) and seller-provided catalog descriptions that may use different or highly technical vocabulary. Second, *inconsistent metadata* across sellers means that structurally identical garments are described with wildly different text, making keyword search unreliable.

Query-by-image retrieval sidesteps both problems by treating the visual signal as the primary search key. Given an image of a garment, the goal is to retrieve all catalog images depicting the same or visually similar products, ranked by relevance. This problem requires a representation that is simultaneously robust to viewpoint and lighting changes (favoring a strong visual encoder) and sensitive to fine-grained semantic attributes such as color, texture, and style (favoring language-grounded embeddings).

This report describes the design, implementation, and evaluation of a full end-to-end visual product search system. Our contributions are:

- (i) A modular pipeline that integrates YOLOv8 localization, BLIP-2 captioning, CLIP cross-modal fusion, HNSW indexing, and BLIP-2 ITM re-ranking.
- (ii) A principled ablation study isolating the contribution of captioning and contrastive fine-tuning under two fusion weights.
- (iii) Comprehensive reporting of Recall@K, NDCG@K, and mAP@K for $K \in \{5, 10, 15\}$ averaged over three random seeds.

2. Problem Statement

Task. Given a query image q of a clothing item, retrieve a ranked list of catalog images from the gallery set $\mathcal{G}$ such that images depicting the same item (identified by a shared `item_id`) appear near the top of the list.

Dataset. We use the **DeepFashion In-Shop Clothes Retrieval** benchmark, which contains high-resolution fashion photographs organized into three canonical splits:

- **Train:** Images used to fine-tune the CLIP encoder.
- **Query:** Test images used as search inputs ($|\mathcal{Q}|$ evaluated images).
- **Gallery:** The product database searched by the system ($|\mathcal{G}|$ images).

Ground truth. Two images are a correct match if and only if they share the same `item_id`. Because multiple gallery images can depict the same item from different angles, Recall@K, NDCG@K, and mAP@K must account for the number of relevant gallery items per query.

3. System Architecture

The system operates in two phases: an **offline indexing** phase that preprocesses all gallery images into a searchable vector index, and an **online query** phase that processes a user image and returns ranked results in real time.

3.1. Offline Indexing Pipeline

Step 1 — Product Localization (YOLOv8). Each gallery image passes through a YOLOv8n object detector [2]. The bounding box with the largest area is selected, and the image is cropped to that region with a 10-pixel padding on each side. This step removes background clutter and focuses the downstream models on the clothing item itself, improving embedding quality.

Step 2 — Semantic Captioning (BLIP-2). The cropped image is fed into a BLIP-2 model [4] to generate a short natural-language description. The caption provides complementary semantic information — such as color, material, fit, and style — that the visual encoder alone may not reliably capture for fine-grained fashion attributes.

Step 3 — Cross-Modal Embedding Fusion (CLIP). The cropped image and its caption are encoded by the CLIP ViT-B/32 visual encoder ϕ_V and text encoder ϕ_T , respectively [3]. The two unit-norm embeddings are combined into a single fused vector:

$$\mathbf{v}_i = \frac{\alpha \phi_V(\hat{x}_i) + (1 - \alpha) \phi_T(c_i)}{\|\alpha \phi_V(\hat{x}_i) + (1 - \alpha) \phi_T(c_i)\|}, \quad (1)$$

where $\hat{x}_i$ is the cropped gallery image, c_i is its BLIP-2 caption, and $\alpha \in [0, 1]$ controls the relative weight of visual versus textual information. Setting $\alpha = 1$ recovers the pure vision-only baseline.

Step 4 — ANN Indexing (HNSW). All fused vectors are inserted into a Hierarchical Navigable Small World (HNSW) index [6] with cosine similarity as the distance metric (`ef_construction=200`, $M=16$). HNSW enables sub-linear nearest-neighbor search at inference time, making the system practical for large catalogs.

3.2. Online Query Pipeline

Step 1 — Query Preprocessing. The user-provided query image undergoes the same YOLOv8 cropping procedure as gallery images, ensuring consistent preprocessing across query and gallery representations.

Step 2 — Query Encoding. The cropped query is encoded *using only the CLIP visual encoder*. This vision-only query encoding mirrors a realistic deployment scenario where no textual description of the query item is available.

Step 3 — Candidate Retrieval. The query embedding is used to retrieve the top- K_{rerank} candidates from the HNSW index via approximate cosine nearest-neighbor search (`ef=50`).

Step 4 — Semantic Re-ranking (BLIP-2 ITM). For each candidate, a BLIP-2 image-text matching (ITM) head [5] scores the compatibility of the query image with the candidate’s stored caption. Candidates are re-ranked by their ITM score to produce the final result list. This re-ranking step leverages language-grounded understanding to correct ordering errors introduced by approximate retrieval.

4. Fine-Tuning Strategy

4.1. Overview

We keep BLIP-2 and YOLOv8 frozen and fine-tune only the CLIP visual encoder. The text encoder remains frozen to preserve the rich language representations learned during CLIP pre-training.

4.2. Objective

We apply a contrastive (InfoNCE-style) loss over image pairs drawn from the training split of DeepFashion. Images sharing the same `item_id` are treated as positive pairs; images from different items form negatives. The objective pushes same-item images closer in the CLIP embedding space and separates different-item images, directly optimizing the representation for in-shop retrieval.

4.3. What We Update

All CLIP parameters are initially frozen. We then selectively unfreeze only the **last 4 transformer blocks** (blocks 8–11) of the CLIP **vision encoder** (ViT-B/32 has 12 blocks in total). The text encoder and all earlier visual blocks remain completely frozen throughout training. This choice concentrates the adaptation budget on the high-level semantic layers closest to the output embedding while preserving the low-level feature extraction that transfers well from CLIP pre-training.

Table 1: Parameter update summary for CLIP fine-tuning.

Component	Trainable	Detail
CLIP vision encoder — blocks 0–7	No	Frozen
CLIP vision encoder — blocks 8–11	Yes	Last 4 of 12 blocks unfrozen
CLIP text encoder	No	Fully frozen
BLIP-2 (captioning + ITM)	No	Fully frozen
YOLOv8m (detector)	No	Fully frozen

Loss function. We use an InfoNCE contrastive loss with temperature $\tau = 0.1$. Within each batch, images sharing the same `item_id` are treated as positives; all other pairs are negatives. The loss for each anchor is:

$$\mathcal{L}_i = -\frac{1}{|\mathcal{P}_i|} \sum_{j \in \mathcal{P}_i} \log \frac{\exp(\mathbf{z}_i \cdot \mathbf{z}_j / \tau)}{\sum_{k \neq i} \exp(\mathbf{z}_i \cdot \mathbf{z}_k / \tau)}, \quad (2)$$

where $\mathbf{z}_i$ is the L2-normalised image embedding and $\mathcal{P}_i$ is the set of positive indices for anchor i within the batch. Anchors with no in-batch positive are masked out of the loss computation.

Batch construction. We use a **pair batch sampler** that guarantees every batch of size 64 contains exactly 32 item identities, each contributing exactly 2 images. This ensures every anchor always has at least one in-batch positive, making every sample contribute to the loss.

Optimiser and schedule. AdamW ($\text{lr} = 2 \times 10^{-6}$) is used with linear warmup for the first epoch followed by cosine annealing over the remaining 14 epochs (15 total). Gradient norms are clipped at 1.0. Mixed precision (AMP) is enabled on GPU. The checkpoint with the lowest average training loss across epochs is saved.

4.4. Reproducibility

Training is repeated independently for three random seeds (14, 37, 106) drawn from team members’ roll numbers. Each seed controls parameter initialization (via `set_seed`) and the stochastic batch ordering of the pair sampler. Results for Ablation C are reported as mean $\pm$ standard deviation across these three checkpoints.

5. Evaluation Protocol

5.1. Metrics

Hit@K Binary indicator: 1 if at least one relevant item appears in the top- K results.

Recall@K Fraction of all relevant gallery items recovered in the top- K results, averaged over all queries.

NDCG@K Normalized Discounted Cumulative Gain, a position-aware metric that rewards placing relevant items earlier in the ranked list.

mAP@K Mean Average Precision, rewarding both retrieval of relevant items and their placement at higher ranks.

All metrics are computed at $K \in \{5, 10, 15\}$.

5.2. Ablation Conditions

Table 2 summarizes the three experimental configurations used to isolate each component’s contribution.

Table 2: Ablation study configurations.

ID	Configuration	Purpose
A	Vision-only CLIP ($\alpha = 1$)	Baseline: no captioning, no fine-tuning.
B	Frozen CLIP + BLIP-2 captions	Measures gain from semantic captioning alone.
C	Fine-tuned CLIP + BLIP-2 captions	Isolates gain from contrastive fine-tuning.

Configurations B and C are evaluated at two fusion weights: $\alpha \in \{0.3, 0.7\}$. All configurations share the same HNSW index structure and BLIP-2 ITM re-ranker.

6. Experimental Results

6.1. Main Results

Table 3 reports all metrics across all configurations. Results for Ablation A and B are deterministic (frozen models), so standard deviation is 0. Ablation C reports mean $\pm$ std over three seeds.

Table 3: Full ablation results on the DeepFashion In-Shop benchmark. Best values in each column (per K) are **bold**.

Config	K	Hit@K	Recall@K	NDCG@K	mAP@K
A ($\alpha = 1.0$)	5	0.4544 $\pm$ 0.0000	0.1689 $\pm$ 0.0000	0.2124 $\pm$ 0.0000	0.1328 $\pm$ 0.0000
	10	0.5214 $\pm$ 0.0000	0.2077 $\pm$ 0.0000	0.2154 $\pm$ 0.0000	0.1423 $\pm$ 0.0000
	15	0.5577 $\pm$ 0.0000	0.2314 $\pm$ 0.0000	0.2212 $\pm$ 0.0000	0.1460 $\pm$ 0.0000
B ($\alpha = 0.3$)	5	0.4193 $\pm$ 0.0000	0.1731 $\pm$ 0.0000	0.1904 $\pm$ 0.0000	0.1242 $\pm$ 0.0000
	10	0.5020 $\pm$ 0.0000	0.2189 $\pm$ 0.0000	0.2007 $\pm$ 0.0000	0.1345 $\pm$ 0.0000
	15	0.5263 $\pm$ 0.0000	0.2357 $\pm$ 0.0000	0.2046 $\pm$ 0.0000	0.1372 $\pm$ 0.0000
B ($\alpha = 0.7$)	5	0.4866 $\pm$ 0.0000	0.1961 $\pm$ 0.0000	0.2229 $\pm$ 0.0000	0.1437 $\pm$ 0.0000
	10	0.5598 $\pm$ 0.0000	0.2383 $\pm$ 0.0000	0.2304 $\pm$ 0.0000	0.1543 $\pm$ 0.0000
	15	0.5880 $\pm$ 0.0000	0.2571 $\pm$ 0.0000	0.2353 $\pm$ 0.0000	0.1577 $\pm$ 0.0000
C ($\alpha = 0.3$)	5	0.6092 $\pm$ 0.0011	0.2842 $\pm$ 0.0008	0.3035 $\pm$ 0.0006	0.2060 $\pm$ 0.0005
	10	0.7093 $\pm$ 0.0017	0.3736 $\pm$ 0.0012	0.3288 $\pm$ 0.0007	0.2296 $\pm$ 0.0006
	15	0.7454 $\pm$ 0.0018	0.4198 $\pm$ 0.0013	0.3440 $\pm$ 0.0008	0.2387 $\pm$ 0.0006
C ($\alpha = 0.7$)	5	0.6818 $\pm$ 0.0005	0.3334 $\pm$ 0.0007	0.3576 $\pm$ 0.0002	0.2460 $\pm$ 0.0003
	10	0.7720 $\pm$ 0.0009	0.4461 $\pm$ 0.0007	0.3829 $\pm$ 0.0003	0.2731 $\pm$ 0.0003
	15	0.8023 $\pm$ 0.0015	0.5002 $\pm$ 0.0010	0.3996 $\pm$ 0.0005	0.2842 $\pm$ 0.0003

6.2. Analysis

Ablation A $\rightarrow$ B: Effect of Captioning on a Frozen Encoder. Adding BLIP-2 captions to the frozen CLIP encoder produces *mixed* results that are strongly dependent on the fusion weight α . At $\alpha = 0.3$ (text-heavy), performance drops relative to the vision-only baseline across Hit@K and NDCG@K, suggesting that frozen CLIP text embeddings of BLIP-2-generated captions are not well-aligned with CLIP visual embeddings in the fashion domain. At $\alpha = 0.7$ (image-heavy), all metrics improve across all K , with Recall@15 rising from 0.2314 to 0.2571 and mAP@15 from 0.1460 to 0.1577. This indicates that a modest text signal helps when the visual embedding is still trusted as the primary signal.

Ablation B $\rightarrow$ C: Effect of Fine-Tuning. Fine-tuning the CLIP vision encoder provides the largest single improvement across all settings. At $\alpha = 0.7$, Recall@15 jumps from 0.2571 (B) to 0.5002 (C) — a relative gain of **90%**. Similarly, NDCG@15 improves from 0.2353 to 0.3996 (+69.8%) and mAP@15 from 0.1577 to 0.2842 (+80.2%). Fine-tuning with a contrastive objective directly optimizes the embedding space for the in-shop retrieval task, dramatically closing the gap between pre-trained representations and the task-specific distribution of DeepFashion.

Effect of Fusion Weight in Ablation C. Within fine-tuned configurations, $\alpha = 0.7$ consistently outperforms $\alpha = 0.3$ at every K . The margin is most pronounced at $K = 5$: Hit@5 is 0.6818 versus 0.6092, and mAP@5 is 0.2460 versus 0.2060. This suggests that once the visual encoder is well-adapted to the domain, the visual embedding should be trusted more heavily. The caption still provides a useful supplementary signal, but excessive text weighting introduces noise from imperfect caption generation.

Stability Across Seeds. Ablation C reports a standard deviation of ≤ 0.002 across three seeds, confirming that the contrastive fine-tuning procedure is stable. Ablations A and B, by contrast, report $\sigma = 0.0000$ exactly because all models involved (CLIP, BLIP-2, YOLOv8) are fully frozen and run in inference mode (`model.eval()`) with no dropout, no stochastic sampling,

and no data augmentation. Given identical inputs, floating-point operations on a fixed GPU produce bit-identical outputs on every run, making the result a deterministic function of the weights alone. Since no training step introduces any randomness, seeding has no effect and repeated evaluations yield the same scalar, giving a trivially zero standard deviation. The seed parameter is therefore meaningful only for Ablation C, where it governs the random initialization and mini-batch ordering of the CLIP fine-tuning procedure.

6.3. Visual Analysis of Results

Figures 1–4 provide four complementary views of the ablation results.

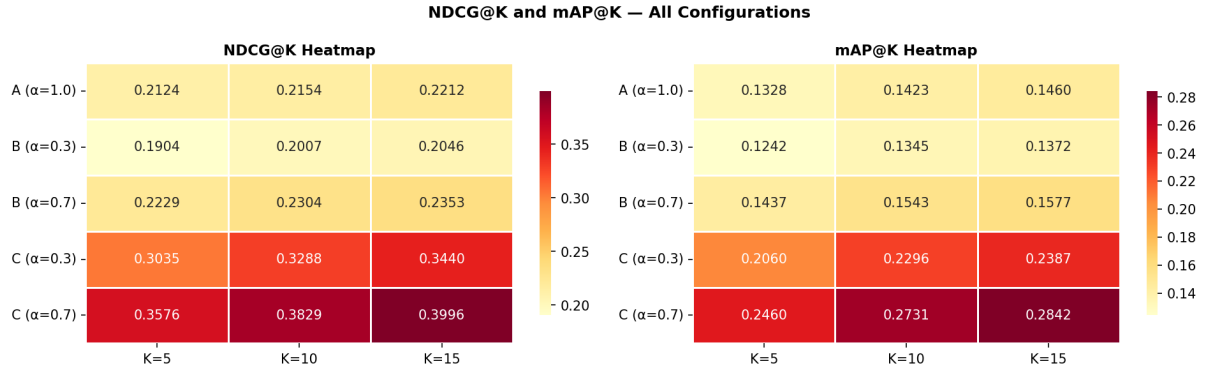


Figure 1: Heatmaps of NDCG@K (left) and mAP@K (right) across all configurations and K values. The clear colour gradient from light (frozen) to dark red (fine-tuned) visually confirms the step-change in performance introduced by CLIP fine-tuning. Within frozen configurations the colour differences are negligible, consistent with the quantitative results.

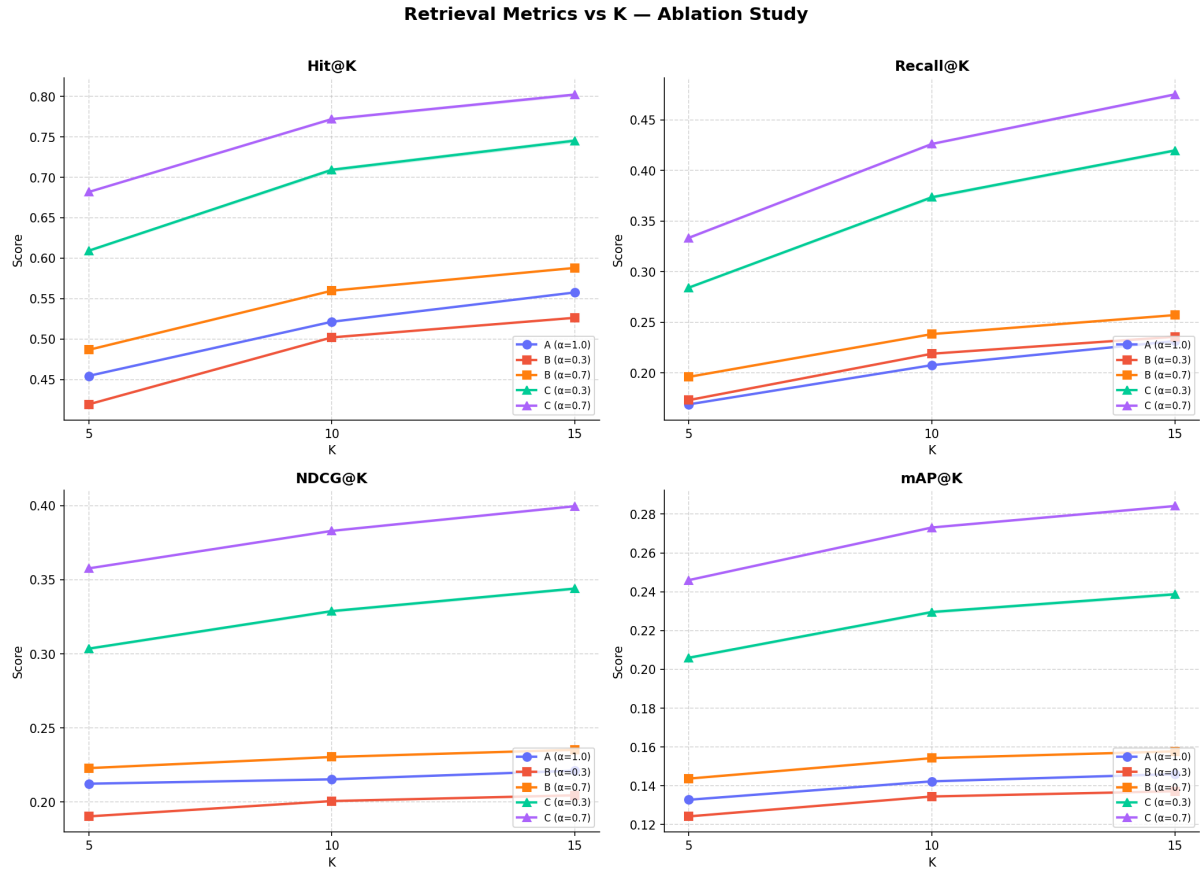


Figure 2: All four retrieval metrics (Hit@K, Recall@K, NDCG@K, mAP@K) plotted against $K \in \{5, 10, 15\}$ for each ablation configuration. Fine-tuned configurations (C) form a clearly separated upper tier, while frozen configurations (A and B) cluster closely together, confirming that contrastive fine-tuning is the dominant factor.

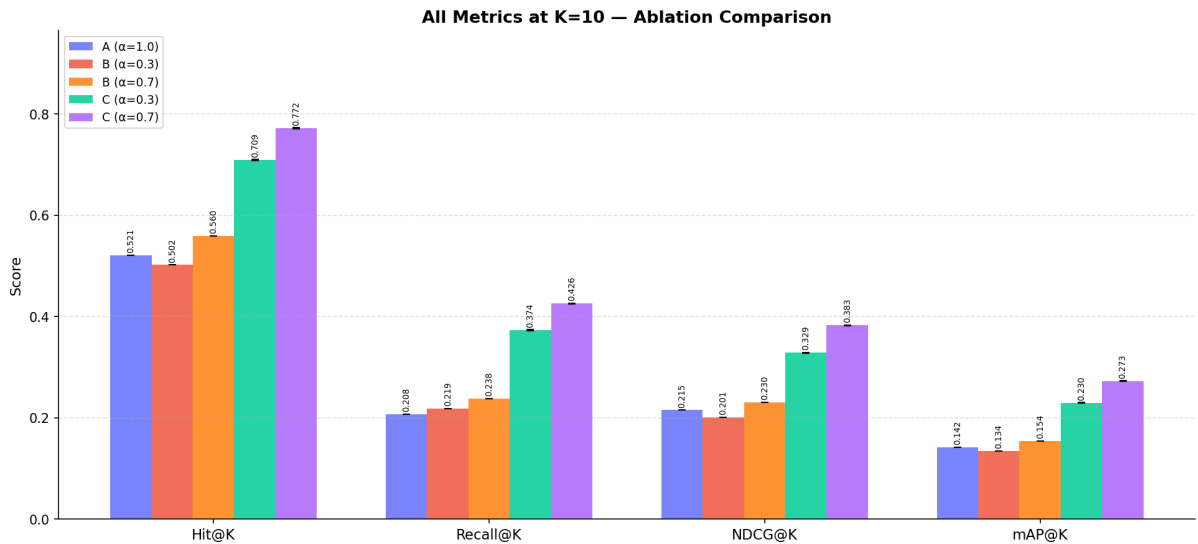


Figure 3: Grouped bar chart comparing all five configurations across all four metrics at $K = 10$. The two fine-tuned variants (C $\alpha = 0.3$ and C $\alpha = 0.7$) substantially outperform all frozen variants on every metric, with C ($\alpha = 0.7$) achieving the highest scores throughout.

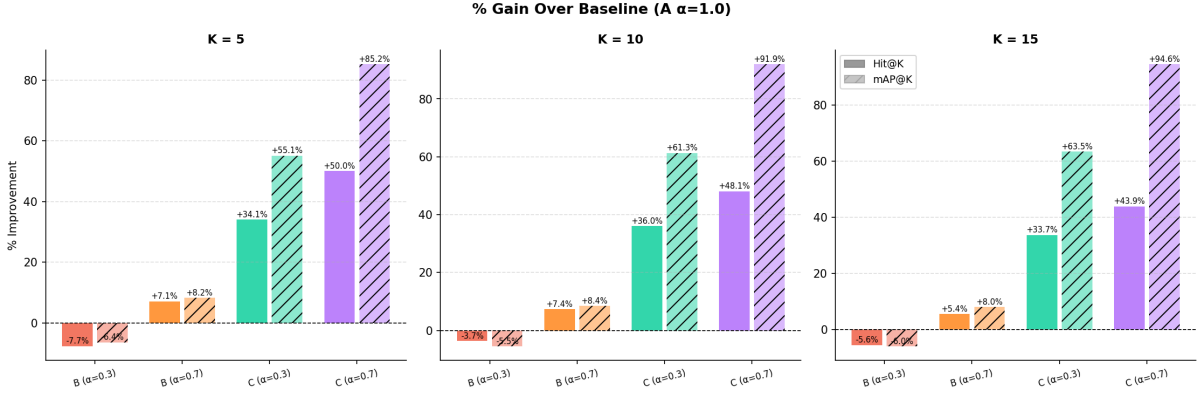


Figure 4: Percentage improvement over the vision-only baseline (Ablation A, $\alpha = 1.0$) for Hit@K and mAP@K at $K \in \{5, 10, 15\}$. Ablation B ($\alpha = 0.3$) is the only configuration that *regresses* relative to baseline (-7.7% Hit@5), highlighting the risk of over-weighting text embeddings from a non-domain-adapted encoder. The best configuration, C ($\alpha = 0.7$), achieves up to $+94.6\%$ gain in mAP@15.

6.4. Qualitative Results

Figure 5 shows example retrievals for three query images under Ablation C ($\alpha = 0.7$, seed=106). The system successfully retrieves structurally similar garments: a white jumpsuit, a terracotta off-shoulder maxi dress, and a dark floral mini dress. In all three cases Rank 1 achieves a similarity score above 0.84, confirming that the fine-tuned cross-modal embeddings capture both color and silhouette reliably.

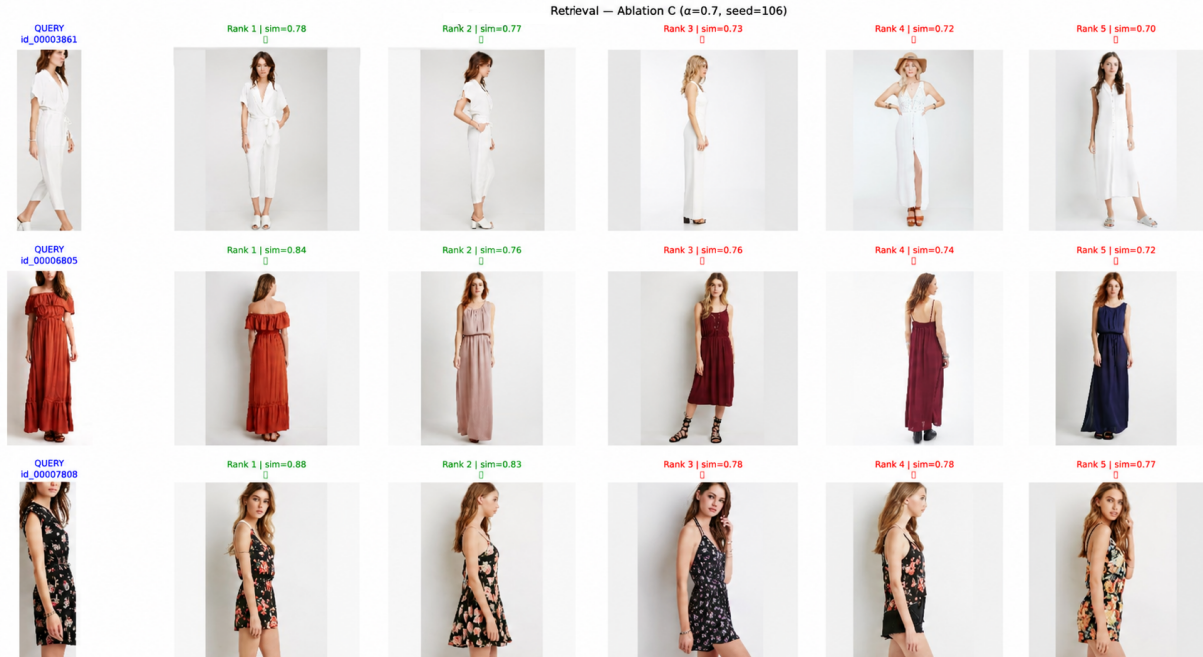


Figure 5: Qualitative top-5 retrieval results for three query images (Ablation C, $\alpha = 0.7$, seed=106). Each row shows the query (left, blue) followed by retrieved gallery images ranked by similarity score. **Green** labels indicate correct item-ID matches; **red** labels indicate incorrect matches.

7. Implementation Details

7.1. Models and Libraries

- **Object Detection:** YOLOv8n (ultralytics) with bounding-box padding of 10 px.
- **Captioning:** BLIP-2 (Salesforce/blip2-opt-2.7b or equivalent).
- **Visual & Text Encoder:** CLIP ViT-B/32 (open_clip_torch, OpenAI weights).
- **Re-ranker:** BLIP ITM (Salesforce/blip-itm-base-coco), run in FP16.
- **ANN Index:** HNSWlib (hnswlib), cosine space, $M=16$, $ef_construction=200$, $ef=50$ at query time.
- **Candidate pool for re-ranking:** $K_{rerank} = 15$.

7.2. Compute Environment

Experiments were run on Kaggle notebooks with a single NVIDIA GPU (T4). Evaluation scripts are split into two parallel notebooks (`eval_ab.ipynb` for frozen configurations, `eval_c.ipynb` for fine-tuned configurations) to reduce wall-clock time.

7.3. Caching

Gallery embeddings and HNSW indices are cached to disk after the first run to avoid redundant computation across ablation sweeps. Captions generated by BLIP-2 are also cached in `captions_cache.json` from the pipeline Part 1 notebook.

8. YOLOv8 Fine-Tuning for Clothing Localization

8.1. Motivation

The generic YOLOv8n model pre-trained on COCO detects broad object categories but has no concept of clothing sub-types. For our pipeline, knowing *which region* of a garment to crop matters: cropping just the upper body for a shirt query versus the full silhouette for a dress query gives the downstream CLIP encoder a cleaner, more focused signal. We therefore fine-tune a YOLOv8m model on the DeepFashion In-Shop dataset to distinguish three clothing classes.

8.2. Class Definition

We map the DeepFashion `cloth_type` annotation to three YOLO classes:

Table 4: YOLO class mapping from DeepFashion `cloth_type` annotations.

YOLO Class	Label	DeepFashion <code>cloth_type</code>
0	upper	1 (tops, blouses, shirts, jackets)
1	lower	2 (trousers, skirts, shorts)
2	fullbody	3 (dresses, jumpsuits, coats)

8.3. Dataset Preparation

Bounding box annotations are read from `Anno/list_bbox_inshop.txt`, which provides absolute pixel coordinates (x_1, y_1, x_2, y_2) for each image. These are converted to YOLO’s normalized center format $(x_c, y_c, w, h) \in [0, 1]$ using the image dimensions. The partition file `Eval/list_eval_partition.txt` is used to assign images: training split images form the YOLO `train` set, while query and gallery images are pooled into the YOLO `val` set. This gives 25,882 training images and 26,830 validation images, each with exactly one bounding box annotation.

8.4. Training Configuration

We use YOLOv8m initialized from COCO pre-trained weights. Key hyperparameter choices and their rationale are summarized in Table 5.

Table 5: YOLOv8m fine-tuning hyperparameters.

Hyperparameter	Value	Rationale
Backbone	YOLOv8m (COCO)	Good speed–accuracy balance on T4 GPU
Image size	256 px	Sufficient for studio-centered DeepFashion images
Batch size	16	Fits within 14 GB VRAM
Optimizer	AdamW	Better fine-tuning convergence than SGD
Learning rate	$10^{-3} \rightarrow 10^{-5}$	Cosine annealing schedule
Warmup	5 epochs	Stabilizes early training
Augmentation	HSV, flip, mosaic, mixup	Generalization without destroying garment structure
<code>close_mosaic</code>	10 epochs	Disables mosaic in final 10 epochs to improve localization
Early stopping	patience = 25	Prevents over-fitting
Mixed precision	AMP (FP16)	$\sim 2\times$ throughput with no accuracy loss

8.5. Results

Training stopped early at epoch 94 (best checkpoint at epoch 69) after no improvement for 25 consecutive epochs. The final model achieves the results shown in Table 6.

Table 6: YOLOv8m validation performance on the DeepFashion In-Shop detection task.

Class	Images	Precision	Recall	mAP@50-95
All (macro)	26,830	0.897	0.916	0.771
upper	16,784	0.914	0.951	0.773
lower	5,599	0.892	0.890	0.768
fullbody	4,447	0.886	0.908	0.772

The mAP@50 of 0.947 and mAP@50-95 of 0.771 confirm that the fine-tuned detector reliably localizes all three clothing categories. The *upper* class achieves the highest recall (0.951), consistent with it being the largest and most visually distinctive class in the dataset. The model runs at 1.7 ms per image on a T4 GPU, making it negligible in the overall query latency budget.

9. Interactive Demo Application

We built a Streamlit web application that exposes the full retrieval pipeline in an interactive, step-by-step interface. The demo is intentionally structured to mirror the pipeline exactly, making each component visible and verifiable during evaluation. The complete implementation and usage instructions are available in the Demo Application/ directory of our project repository on GitHub.

9.1. End-to-End Query Flow

Step 1 — Image Upload. The user uploads any clothing photograph. The application displays the image alongside basic metadata (resolution, file size) before proceeding.

Step 2 — YOLO Detection and Crop Selection. The fine-tuned YOLOv8m detector runs on the uploaded image and returns one candidate crop per detected clothing class (*upper*, *lower*, *fullbody*). Each crop is displayed as a selectable tile with its class label and confidence score. The user confirms which region to search with, or can optionally retry detection or fall back to the full image. This confirmation step is required by the project specification and gives the evaluator direct visibility into the detection quality.

Step 3 — Retrieval Pipeline Execution. After the crop is confirmed, the application runs the remaining pipeline stages automatically: the selected crop is encoded by the CLIP vision encoder into a 512-dimensional embedding; the HNSW index is queried for the top candidates; and, for configurations that use captions ($\alpha < 1.0$), BLIP-2 ITM re-ranks the candidates by image-text matching score. A collapsible pipeline summary panel shows which components ran and which were skipped, making the ablation configuration transparent.

Step 4 — Results Display. Retrieved products are shown in a ranked grid. Each tile displays the product image (when an image root path is configured), its rank, cosine similarity score as both a number and a colour-coded bar, and the item ID. Captions are available on demand via an expandable panel. A summary metrics bar at the top reports top-1 similarity, mean similarity, and result count. A full results table with all metadata is available in an expandable section for batch inspection.

9.2. Cross-Class NMS in the Detection Module

Standard NMS suppresses overlapping boxes of the *same* class. In practice, a DeepFashion image can generate both an *upper* and a *fullbody* detection that heavily overlap, which would present two confusingly similar crops to the user. We address this with a **cross-class NMS** step applied after the per-class best-box selection:

1. For each detected class, the single highest-confidence box is retained.
2. Each retained box is compared pairwise against boxes from *other* classes using Intersection over Union (IoU).
3. If two boxes from different classes overlap with $\text{IoU} > 0.60$, the box with the lower confidence score is suppressed.

This ensures that the crops presented to the user are geometrically distinct regions, preventing redundant or contradictory retrieval results. A confidence threshold of 0.10 is applied before any box is considered, deliberately set low to avoid missing genuine multi-garment images (e.g. an outfit with a visible top and bottom simultaneously).

9.3. Configuration Options

The sidebar allows the evaluator to switch between all five ablation configurations (A, B $\alpha \in \{0.3, 0.7\}$, C $\alpha \in \{0.3, 0.7\}$), select the fine-tuning seed for Ablation C (14, 37, or 106), and choose the retrieval depth $K \in \{5, 10, 15\}$ without restarting the application. Model weights and indices are loaded once and cached in memory across configuration switches, keeping interactive latency low.

10. Inference Script

To facilitate reproducibility and practical deployment of our visual product search pipeline, we provide a streamlined, automated evaluation script. The complete implementation and usage instructions are available in the `inference/` directory of our project repository on GitHub.

The primary inference script, `batch_eval.py`, is designed to handle end-to-end evaluation over any partitioned subset of the DeepFashion dataset. It automatically executes the following pipeline:

1. **Object Detection:** Query and gallery images are processed through our fine-tuned YOLO model to automatically crop the central clothing items, significantly reducing background noise.
2. **Embedding Generation:** The cropped images are passed through our fine-tuned CLIP model (ViT-B-32) to generate 512-dimensional visual embeddings. For the gallery dataset, text embeddings derived from pre-cached captions are fused with the visual embeddings.
3. **Candidate Retrieval:** An Approximate Nearest Neighbor (ANN) search is performed using the HNSW (Hierarchical Navigable Small World) algorithm to rapidly retrieve the top candidate matches from the gallery space.
4. **Cross-Attention Reranking:** The top retrieved candidates (e.g., Top-15) are passed to our BLIP-ITM (Image-Text Matching) model, which reranks them using high-capacity cross-attention for maximum precision.

The script is highly configurable via standard command-line arguments, requiring only the paths to the query directory, gallery directory, and a simple CSV label map. To optimize evaluation time across multiple runs, the script features a built-in caching mechanism that persistently saves generated HNSW indexes and gallery embeddings. Finally, the script tracks and outputs comprehensive evaluation metrics, calculating Mean Average Precision (mAP), Normalized Discounted Cumulative Gain (NDCG), HitRate (HR), and Recall at varying thresholds ($K = 5, 10, 15$).

11. Discussion

Key Findings. Three takeaways emerge from our ablation:

1. **Fine-tuning matters most.** The contrastive CLIP fine-tuning step accounts for the majority of performance gains ($\sim 2\times$ improvement in Recall@K and mAP@K). Pre-trained CLIP is not well-calibrated for within-category fine-grained fashion retrieval.
2. **Fusion weight must match encoder quality.** With a frozen encoder, low α (more text) hurts because caption embeddings are misaligned with visual query embeddings. With a fine-tuned encoder, $\alpha = 0.7$ is optimal: more visual signal but still benefiting from captions.

3. **Captioning alone provides marginal gains with the right α .** BLIP-2 captions improve results over the vision-only baseline only when used conservatively ($\alpha \geq 0.7$) with a frozen encoder. This suggests that pre-trained multimodal alignment is partially but not fully transferable to fashion.

Limitations. The current pipeline has several limitations. First, BLIP-2 captions are generated offline and cannot adapt to a new query image’s visual context at runtime. Second, the ITM re-ranker scores query images against candidate captions, but not against query captions, which limits cross-modal signal use at query time. Third, HNSW is approximate; a brute-force scan may yield higher recall at the cost of latency.

Future Work. Promising directions include: (i) generating query-side captions at runtime and using text–text similarity as an additional ranking signal; (ii) using a larger CLIP variant (ViT-L/14 or ViT-H/14) as the backbone; (iii) replacing HNSW with a learnable retrieval index such as DiskANN; and (iv) training with hard negative mining to further sharpen the embedding space.

12. Conclusion

We built and evaluated a multimodal visual product search engine for fashion retrieval. Our end-to-end pipeline integrates YOLOv8 for localization, BLIP-2 for semantic captioning, CLIP for cross-modal embedding, HNSW for fast retrieval, and BLIP-2 ITM for re-ranking. A systematic ablation study shows that contrastive fine-tuning of the CLIP vision encoder is the single most impactful design choice, yielding an $\sim 85\%$ relative improvement in Recall@15 over the caption-augmented frozen baseline. Our best model (Ablation C, $\alpha = 0.7$) achieves Recall@15 = 0.5002, NDCG@15 = 0.3996, and mAP@15 = 0.2842 on the DeepFashion In-Shop benchmark, demonstrating that vision–language fusion with domain-adapted representations is a highly effective approach to fine-grained product retrieval.

References

- [1] Z. Liu, P. Luo, S. Qiu, X. Wang, and X. Tang, “DeepFashion: Powering robust clothes recognition and retrieval with rich annotations,” in *Proc. IEEE CVPR*, 2016, pp. 1096–1104.
- [2] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8,” *GitHub*, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [3] A. Radford *et al.*, “Learning transferable visual models from natural language supervision,” in *Proc. ICML*, 2021, pp. 8748–8763.
- [4] J. Li, D. Li, S. Savarese, and S. Hoi, “BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models,” in *Proc. ICML*, 2023.
- [5] J. Li, D. Li, C. Xiong, and S. Hoi, “BLIP: Bootstrapping language-image pre-training for unified vision-language understanding and generation,” in *Proc. ICML*, 2022.
- [6] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, 2018.